CEDT Lab 7 - Communication Protocol (1)

จุดประสงค์การเรียนรู้

1. เข้าใจการติดต่อสื่อสารระหว่าง CPU ใน Microcontroller กับ Peripheral
2. เข้าใจการทำงานผ่านทาง Serial Terminal

อุปกรณ์ที่ใช้ในการทดลอง

1. ตัวต้านทานค่าต่างๆ (220 Ohm , 1k Ohm, 10kOhm)
2. LEDs (Diode)
3. สายไฟ
4. ESP32
5. มัลติมิเตอร์
6. Switch หรือ ปุ่ม

UART

Serial Communication / U(S)ART

Serial communication is a method for using a sequence of bits to transfer a byte of data. This is in contrast to parallel communication, where several bits are sent as a whole, on a link with several parallel channels as Illustration.

For universal asynchronous receiver/transmitter (UART) – Universal Synchronous/Asynchronous Receiver/Transmitter (USART) also supports synchronous operation, we use 8 bit to represent a byte of data. A byte is generally equal to a character. Thus, there are (theoretically) 256 possible characters in a byte. One standard that provides association between a value and a physical character is ASCII (see next section for more details).

With serial communication, communication between two devices can be done with just two wires (one for sending and one for receiving). This kind of communication is cheaper compared to parallel communication where 8 wires are needed to transmit a byte of data. Thus, to transfer a byte of data from one device to another, we have to send (at least) 8 times (one for each bit). Given that no clock or control signal is involved in the communication, the key critical to the success of such communication is timing. Both devices has to be configured for the same speed of clock (specified as bit per second or bps) and the same protocol in order to observe the right data at the right time. This is the basic of network communication that we used today.

Bit number	1	2	3	4	5	6	7	8	9	10	11	12
	Start bit	**5–9 data bits**									**Stop bit(s)**	
	Start	Data 0	Data 1	Data 2	Data 3	Data 4	Data 5	Data 6	Data 7	Data 8	Stop	

https://en.wikipedia.org/wiki/Universal_asynchronous_receiver/transmitter

ASCII

ASCII is short for the American Standard Code for Information Interchange. It is a character-encoding scheme based on the English alphabet, numbers and punctuations. It is nowadays used by several digital devices as a way to represent code for characters. Thus, when type a character 'A', the digital computer system simply records an ASCII code to the memory.

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

http://www.infocellar.com/binary/ascii-ebcdic.htm

การทดลอง

1. **ESP32 to Computer with UART**

   ESP32 module มี UART ต่อผ่าน USB-Serial ทำให้คุณสามารถใช้ Serial.print ในการส่งข้อมูลเข้าออก และ Upload program ผ่าน Serial terminal ได้

   UART มี Baudrate ที่สามารถตั้งได้แนะนำให้ตั้งเป็นค่าอย่างน้อย 115200

   ในการทดลองนี้ ให้ต่อวงจรและโปรแกรมทำงานเหมือนใน Lab 4.2 โดยให้เพิ่มการทำงานดังต่อไปนี้

   - เมื่อกดปุ่ม ให้ส่งไปใน Serial Terminal ว่า “Button Pressed” ตามด้วยจำนวนครั้งที่กดเช่น “Button Pressed 1”, (debounced ด้วย)
   - จาก serial terminal เมื่อ user พิมพ์ว่า “on” ให้ ไปติด เมื่อ พิมพ์ว่า “off” ให้ไฟดับโดยการทำงานของปุ่มจะยังถูกต้อง (หาก ไฟดับเพราะคำสั่ง off แล้วมีการกดปุ่มให้เปิดไฟ)

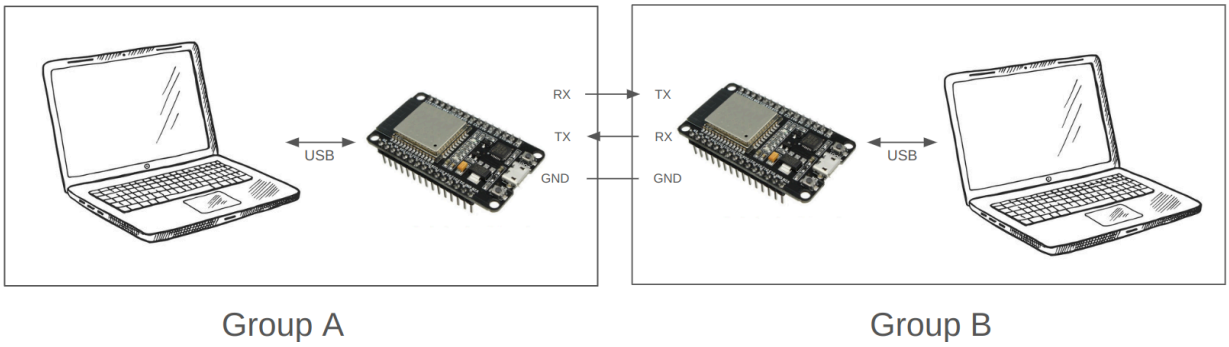
   ส่งใน MCV

- Video และ Code การทำงานของโปรแกรมคุณ

2. **ESP32 to ESP32**

   ให้กลุ่มสองกลุ่มจับกันเป็น สองกลุ่ม โดยให้การทำงานเป็นดังต่อไปนี้

   - กำหนดให้ มี ESP32 สองตัว A และ B เริ่มมาจาก Lab ข้อ 1
   - ต่อสายไฟสามเส้น
     - ESP32 A - TX to ESP32 B - RX
     - ESP32 A - RX to ESP32 B - TX
     - ESP32 A - GND to ESP32 B - GND
     - RX กับ TX เป็น Pin I/O อะไรก็ได้ ที่ไม่ใช่ D2 , D15 ที่ใช้กับปุ่มและ LED



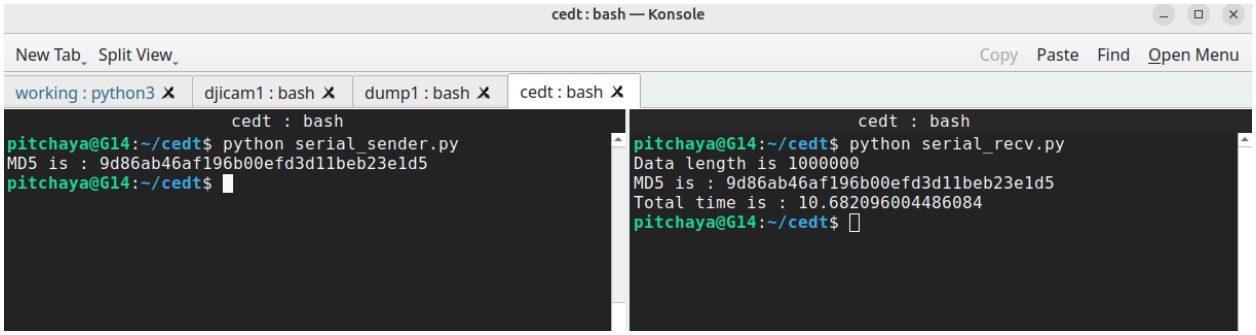
   - ใช้ Software Serial (https://github.com/plerup/espsoftwareserial/) เพื่อให้ สอง Board คุยกัน โดยถ้าพิมพ์จากคอมพิวเตอร์ที่ต่ออยู่กับ ESP32-A ผ่าน USB Serial ให้ ข้อมูลส่งไปที่คอมพิวเตอร์ที่ ESP32-B ต่ออยู่ และถ้าพิมพ์จากคอมพิวเตอร์ที่ต่ออยู่กับ ESP32-B ผ่าน USB Serial ให้ ข้อมูลส่งไปที่คอมพิวเตอร์ที่ ESP32-A ต่ออยู่
   - ให้สามารถทำงานเหมือนในข้อ 1 อยู่ได้ โดยถ้าพิมพ์ on ในเครื่อง A หรือ B ให้ ไฟติดใน LED ที่ต่อกับ ESP32-A และ ESP32-B และ ถ้าพิมพ์ off ในเครื่อง A หรือ B ให้ ไฟดับใน LED ที่ต่อกับ ESP32-A และ ESP32-B

   ส่งใน MCV

- Video และ Code การทำงานของโปรแกรมคุณ

3. **Robust Communication**

- ให้สร้างระบบคล้ายๆกับข้อสองแต่ไม่ต้องเปิดปิดไฟ ให้สนใจแค่การส่งข้อมูลจากเครื่อง Computer A ผ่าน USB Serial ไปยัง ESP32-A ผ่าน UART ไปยัง ESP32-B แล้วไปถึงเครื่อง B (ทางเดียว). โดยต้องส่งข้อมูลทั้งหมด 1,000,000 bytes ที่สร้างด้วยการสุ่มมา โดยไม่มีข้อผิดพลาด (สามารถใช้ Hash เช่น MD5 เพื่อตรวจสอบได้)
- ทดลองการทำงานด้วยโปรแกรม serial_sender.py และ serial_recv.py (อยู่ใน MCV)



- สามารถเปลี่ยน baudrate ในการทำงานได้
- ควรเปลี่ยน N ให้ ค่าน้อยๆ เพื่อการทดลอง
- สามารถเปลี่ยน Protocol ที่ส่งระหว่าง ESP32 จาก UART เป็นอย่างอื่นได้

ส่งใน MCV

- Video และ Code การทำงานของโปรแกรมคุณในการส่งข้อมูล 1,000,000 bytes
- พยายามให้เร็วที่สุดเท่าที่เร็วได้